

Langfuse + Spring AI + PetClinic

Complete installation and integration record for the local Langfuse observability setup used with the Spring AI PetClinic application.

Scope: Langfuse installation, Docker Compose, PostgreSQL port adjustment, project/API keys, Maven dependencies, application.properties, shell exports, startup/test procedure, troubleshooting, and verification of the PetClinic tool-calling trace.

Environment: Ubuntu; PetClinic at ~/dev/spring-petclinic-ai; Spring Boot 4.0.0-SNAPSHOT; Spring AI 2.0.0-M1; Ollama at localhost:11434; qwen2.5:0.5b-instruct; Langfuse locally at localhost:3000.

1. Why Langfuse was installed

The original problem was Spring AI tool calling. A request such as “Give me all PetClinic owners” could sometimes result in the model saying it could not find or use the tools. Langfuse was installed so we could inspect the complete interaction: prompt, model generation, tool call, tool result and final answer.

2. Install Langfuse locally with Docker Compose

```
cd ~/dev
git clone https://github.com/langfuse/langfuse.git langfuse-local
cd ~/dev/langfuse-local/langfuse
```

```
docker compose up -d
docker compose ps
```

The actual Langfuse repository used was ~/dev/langfuse-local/langfuse. The local Langfuse UI is <http://localhost:3000>. The Compose stack runs Langfuse Web/Worker and supporting PostgreSQL, ClickHouse, Redis and MinIO services.

3. PostgreSQL port conflict

PetClinic already used host port 5432, so the Langfuse PostgreSQL host port was changed from 5432 to 5433. The container still uses PostgreSQL port 5432.

```
postgres:
  image: docker.io/postgres:${POSTGRES_VERSION:-17}
  restart: always
  # ...
  ports:
    - "127.0.0.1:5433:5432"
```

```
docker compose config | grep -A25 "^ postgres:"
docker compose ps
```

Final mapping: **127.0.0.1:5433 → container:5432**. Do not change PetClinic's existing database port just for Langfuse.

4. Create Langfuse project and API keys

Open <http://localhost:3000>, create the local account/organization if necessary, create a project, then use the project API-key screen to obtain the public and secret keys.

```
LANGFUSE_PUBLIC_KEY="pk-XXXXXXX"
LANGFUSE_SECRET_KEY="sk-XXXXXXX"
LANGFUSE_BASE_URL="http://localhost:3000"
```

The real secret key is intentionally not recorded here. Never commit it to Git or paste it into chat.

5. Maven dependencies added/verified

The PetClinic project already had Spring AI Ollama, Spring AI vector-store and Spring Boot Actuator. For tracing, the dependency tree showed:

```
io.micrometer:micrometer-tracing-bridge-otel:1.6.0
io.opentelemetry:opentelemetry-exporter-otlp:1.55.0
```

We also added the OpenTelemetry Spring Boot starter. Maven initially failed because the starter version was not managed. We therefore imported the OpenTelemetry instrumentation BOM and kept the starter dependency versionless:

```
<!-- dependencyManagement -->
<dependency>
  <groupId>io.opentelemetry.instrumentation</groupId>
  <artifactId>opentelemetry-instrumentation-bom</artifactId>
  <version>2.28.1</version>
  <type>pom</type>
  <scope>import</scope>
</dependency>

<!-- dependencies -->
<dependency>
  <groupId>io.opentelemetry.instrumentation</groupId>
  <artifactId>opentelemetry-spring-boot-starter</artifactId>
</dependency>
```

After this correction, the Maven project compiled and the tracing dependencies were present.

6. application.properties changes

The existing Ollama configuration was kept. These observability/tracing properties were added:

```
# Ollama
spring.ai.ollama.base-url=http://localhost:11434
spring.ai.ollama.chat.options.model=qwen2.5:0.5b-instruct

# Spring AI observations
spring.ai.chat.observations.log-prompt=true
spring.ai.chat.observations.log-completion=true

# Capture all traces during local debugging
management.tracing.sampling.probability=1.0

# OTLP tracing destination
management.otlp.tracing.endpoint=http://localhost:3000/api/public/otel
management.otlp.tracing.headers.Authorization=Basic ${LANGFUSE_AUTH}
management.otlp.tracing.headers.x-langfuse-ingestion-version=4
```

Prompt/completion observation logging was enabled for local debugging. Sampling was set to 1.0 so test traces are not sampled out.

7. Shell environment used to start PetClinic

We chose shell exports for the test. The important point is that the variables must be present in the environment of the PetClinic JVM.

```
export LANGFUSE_PUBLIC_KEY="pk-XXXXXXX"
export LANGFUSE_SECRET_KEY="sk-XXXXXXX"

export LANGFUSE_AUTH=$(printf '%s:%s' \
  "$LANGFUSE_PUBLIC_KEY" \
  "$LANGFUSE_SECRET_KEY" | base64 -w 0)

export OTEL_EXPORTER_OTLP_ENDPOINT="http://localhost:3000/api/public/otel"
export OTEL_EXPORTER_OTLP_HEADERS="Authorization=Basic $LANGFUSE_AUTH,x-langfuse-ingestion-version=4"

echo "Langfuse public key: ${LANGFUSE_PUBLIC_KEY:0:8}..."
echo "Langfuse secret key: ${LANGFUSE_SECRET_KEY:0:8}..."
echo "Langfuse auth: SET"
echo "OTEL endpoint: $OTEL_EXPORTER_OTLP_ENDPOINT"
echo "OTEL headers: SET"
```

Then PetClinic was started from that same terminal so it inherited the exports:

```
cd ~/dev/spring-petclinic-ai
./mvnw spring-boot:run
```

If PetClinic is launched from IntelliJ instead, terminal exports may not be inherited unless configured in the IntelliJ run environment.

8. Dependency verification commands

```
cd ~/dev/spring-petclinic-ai

./mvnw dependency:tree | grep -Ei "micrometer-tracing|opentelemetry|otlp"

./mvnw dependency:tree | grep -Ei "opentelemetry-spring-boot|starter-actuator|spring-boot-starter-actuator"
```

The dependency tree confirmed the Micrometer tracing bridge, OpenTelemetry SDK/exporter components, OTLP exporter, Spring Boot Actuator and OpenTelemetry Spring Boot starter.

9. The 404 confusion

Opening <http://localhost:3000/api/public/otel> directly in a browser returned 404. This did **not** mean tracing was broken. The URL is an OTLP ingestion endpoint, not a browser page. The application sends telemetry to it programmatically.

PetClinic also logged a warning similar to “Failed to export logs ... HTTP status code 404”. That warning referred to the OpenTelemetry logs signal. The important fact is that the trace signal was successfully ingested: traces appeared in the Langfuse project. Therefore, do not treat that logs warning as proof that tracing is broken.

10. Test the PetClinic chat

Use the existing PetClinic **POST /chat** endpoint. The exact JSON body is determined by the project controller and should not be invented in this guide.

For the integration test, the request was a natural-language question asking for all PetClinic owners. The successful response returned the complete owner list.

11. What the successful trace demonstrated

```
User
|
| Give me all PetClinic owners
v
Spring AI ChatClient
|
v
Ollama / Qwen
|
| tool call: listOwners
v
Spring AI tool execution
|
v
PetclinicTools.listOwners()
|
| real PetClinic owner data
v
Tool result returned to model
|
v
Ollama / Qwen final response
|
v
Langfuse trace
```

This is the core reason Langfuse was installed: when tool calling fails, the trace can show whether the model did not request the tool, whether a tool call was produced, whether the tool executed, and what happened afterward.

12. Final working configuration

Component	Value
PetClinic	~/dev/spring-petclinic-ai
Spring Boot	4.0.0-SNAPSHOT
Spring AI	2.0.0-M1
Ollama	http://localhost:11434
Model	qwen2.5:0.5b-instruct
Langfuse UI	http://localhost:3000
Langfuse PostgreSQL host port	5433
Trace path	Spring AI → Micrometer/OpenTelemetry → OTLP → Langfuse
Sampling	1.0 for local debugging

13. Quick recovery checklist

```
# Start Langfuse
cd ~/dev/langfuse-local/langfuse
docker compose up -d
docker compose ps

# Set project keys
export LANGFUSE_PUBLIC_KEY="pk-XXXXXXX"
export LANGFUSE_SECRET_KEY="sk-XXXXXXX"

# Build Basic Auth
export LANGFUSE_AUTH=$(printf '%s:%s' \
"$LANGFUSE_PUBLIC_KEY" \
"$LANGFUSE_SECRET_KEY" | base64 -w 0)

# OTLP environment
export OTEL_EXPORTER_OTLP_ENDPOINT="http://localhost:3000/api/public/otel"
export OTEL_EXPORTER_OTLP_HEADERS="Authorization=Basic $LANGFUSE_AUTH,x-langfuse-ingestion-version=4"

# Start PetClinic from the same shell
cd ~/dev/spring-petclinic-ai
./mvnw spring-boot:run

# Send POST /chat, then inspect http://localhost:3000
```

14. Key lessons

- The browser 404 on the OTLP endpoint is not a tracing health check.
- Langfuse PostgreSQL uses host port 5433 because PetClinic already uses 5432.
- Spring AI observation settings alone are not the complete export pipeline; Micrometer/OpenTelemetry and OTLP export are involved.
- The OpenTelemetry Spring Boot starter required the OpenTelemetry instrumentation BOM in this Maven setup.
- Shell exports must reach the JVM that starts PetClinic.
- The final owner-list test proved that the local model, Spring AI tool registration/execution, PetClinic data and Langfuse observability were working together.

End of integration record